A FRIENDLY, STEP-BY-STEP INSTALLATION MANUAL

Run the AI Code Generator on your PC

Start from the basics: install the tools, copy the project from GitHub, prepare Python, launch the local website, and optionally connect an AI chat provider.

What this guide is for

Follow these steps on a personal Windows computer to run the project locally. macOS and Linux command alternatives are included. You do not need to know Git or Python beforehand.

PROJECT REPOSITORY

https://github.com/pranavparmaj/ai-code-generator

The project creates starter Flask applications from written prompts. It runs on your computer at http://127.0.0.1:5000; it is not a public website.

You need	Optional
A computer running Windows 10/11, macOS, or Linux; internet access for setup; Python; Git.	A Gemini API key for Gemini chat, or Ollama installed locally for Ollama chat.

Prepared for the AI Code Generator project | October 2026

1. What you are installing

This repository contains a small local web application. Your browser displays the interface; a Python Flask server runs behind it and generates project files on your computer.

Part	What it does
Browser interface	Shows the prompt workspace and talks to the local server.
Python / Flask backend	Parses your prompt, assembles a starter Flask app, runs checks, and exports a ZIP.
Templates and snippets	Reusable HTML templates and short code examples used during generation.
Generated output	New projects and ZIP downloads are saved locally under generated_projects/.
Chat providers	Gemini uses an API key; Ollama uses a model running through a local Ollama service. Both are optional for project generation.

A few terms in plain language

- Repository (repo): the project files stored on GitHub.
- Clone: download a working copy of the repository using Git. This also lets you receive updates later.
- Terminal: a text window where you enter commands. Windows calls it PowerShell or Command Prompt; macOS and Linux use Terminal.
- Python environment: a private folder of Python packages for this project. The .venv folder keeps these packages separate from other programs.
- Localhost: an address that points back to your own computer. Other people cannot open your local app using this address.

You do not need

Node.js, a database server, or a cloud account just to clone and run the main project-generation flow. The bundled requirements install the Python packages used by the application.

How much time? What is downloaded?

Plan for roughly 15-30 minutes if Python and Git are not installed yet. Python packages need an internet connection. The optional Ollama model can require a much larger download and depends on your computer's memory and speed.

2. Install Python and Git

Do this once on the computer. Use the official downloads below, then open a new terminal after installation.

Install Python

Download Python from python.org/downloads. Choose a supported Python 3 release. Python 3.11 or 3.12 is a conservative choice for package compatibility.

- Windows: run the installer. If the installer offers an option to add Python to PATH, enable it. The Python launcher command py should also be available.
- macOS: install the macOS package from python.org, or use the Python distribution your organization already supports.
- Linux: use your distribution's package manager to install Python 3, its venv support, and pip. Package names vary by distribution.

Confirm Python is available

Open PowerShell (Windows), Terminal (macOS/Linux), and enter:

```
py --version
py -m pip --version
```

On macOS or Linux, use python3 in place of py:

```
python3 --version
python3 -m pip --version
```

> **Expected result**
> You should see a Python 3 version and a pip version. If the command is not recognized, close and reopen the terminal. On Windows, rerun the Python installer and ensure the Python launcher is selected.

Install Git

Download Git from git-scm.com/downloads. Install the version for your operating system. The default installation choices are fine for this project.

Confirm it works:

```
git --version
```

You should see a version string beginning with git version. GitHub sign-in is not normally needed to clone this public repository.

3. Clone the project from GitHub

Cloning creates a folder with the project files. First decide where you want that folder, such as Documents or a development folder.

Windows - PowerShell

Open PowerShell. Move to a parent folder where you want the project to live. This example uses Documents:

```
cd $HOME\Documents
git clone https://github.com/pranavparmaj/ai-code-generator.git
cd ai-code-generator
Get-ChildItem
```

macOS or Linux - Terminal

```
cd ~/Documents
git clone https://github.com/pranavparmaj/ai-code-generator.git
cd ai-code-generator
ls
```

What each command did

Command	Meaning
cd ...	Move into a folder in the terminal.
git clone URL	Copy the GitHub repository into a new ai-code-generator folder.
cd ai-code-generator	Enter the project folder so later commands run in the right place.
Get-ChildItem / ls	List the folder contents. You should see backend, frontend, data, templates, snippets, README.md, and requirements.txt.

Important

Keep this terminal open and run the remaining commands from the ai-code-generator folder unless a step says to enter backend. Do not run commands from inside a random folder or from the GitHub website.

Alternative: GitHub Desktop

If your friend prefers buttons to commands, install GitHub Desktop from desktop.github.com, sign in if prompted, choose File > Clone repository, enter the repository URL above, choose a local folder, and click Clone. Then open a terminal in the cloned ai-code-generator folder and continue with the Python setup below.

Official GitHub Desktop download: desktop.github.com.

4. Create the project Python environment

A virtual environment is a project-specific place for installed Python libraries. Create it inside the project folder. You only need to create it once per clone.

Windows PowerShell - recommended

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

After activation, PowerShell usually shows (.venv) at the beginning of the prompt. Now update pip and install the project dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks activation

This policy change applies only to the current PowerShell window. Then activate the environment again:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

If your computer is managed by an organization, follow its policy. You can instead use Command Prompt:

```
py -3 -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip
pip install -r requirements.txt
```

macOS or Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

What the package install does

pip reads requirements.txt and installs Flask (the web server framework), the Gemini client library, and requests (used for local HTTP calls and preview features). The Gemini library is present even if you do not configure a key. This installation does not install Ollama or download an AI model.

> **If installation fails**
> Check that the virtual environment is active, that internet access is available, and that Python is a supported 3.x version. Copy the full first error message; the last line alone may not identify the problem.

5. Start the app

The application must be started from the backend folder because its Python modules are imported by their local names.

Start from the project folder

If you are still in the repository's top-level folder and the environment is active, enter:

```
cd backend
python app.py
```

You should see a message showing the local address. Leave this terminal window open while using the app. The server runs in this window.

Open the app in a browser

Open Chrome, Edge, Firefox, or another browser and go to http://127.0.0.1:5000. The page is served from the computer running the command.

Stop the app

Return to the terminal window where the server is running and press Ctrl+C. This stops the local server. Close the terminal after it stops.

> **Keep the terminal open**
> If the terminal is closed, the server stops and the browser page will no longer connect. Start it again by opening a terminal in the cloned project, activating .venv, running cd backend, and running python app.py.

Try a first generation

- Enter a clear request, for example: Create a contact form with name, email, subject, and message fields.
- Choose any options the workspace offers, then submit the prompt.
- Review the generated preview and validation summary in the page.
- Download the ZIP or find the project folder under the repository's generated_projects folder.

The generation flow can run without an API key or Ollama. The optional chatbot is a separate feature and needs one of those services.

6. Optional: enable Gemini chat

Gemini chat calls Google's Gemini API. Each person should use their own Google AI Studio account and key. Never share your key with someone else, put it in the source code, or commit it to GitHub.

Create and set a key

Create or manage a key in Google AI Studio API Keys. Google recommends keeping API keys out of source code and using an environment variable. For this project, the variable name is GEMINI_API_KEY.

Set the variable in the same terminal window that will run the server.

Windows PowerShell

```
$env:GEMINI_API_KEY = "paste-your-own-key-here"
cd $HOME\Documents\ai-code-generator\backend
python app.py
```

Use the actual location of the cloned folder in the cd command. The key only remains set in this PowerShell session. If you close the window, set it again next time.

Windows Command Prompt

```
set GEMINI_API_KEY=paste-your-own-key-here
cd %USERPROFILE%\Documents\ai-code-generator\backend
python app.py
```

macOS or Linux

```
export GEMINI_API_KEY="paste-your-own-key-here"
cd ~/Documents/ai-code-generator/backend
python app.py
```

What if Gemini is not configured?

The app's main project generation still works. If the chatbot is set to Gemini and no key is configured, it responds that GEMINI_API_KEY is required. Setting a key is optional and may be subject to Google account terms, quota, or billing settings.

> **Key safety**
> Do not put a real key in a .py file, README, screenshot, email, or public chat. Do not ask the project owner for their key. If a key is exposed, revoke it in its provider console and create a replacement.

Official security guidance: ai.google.dev/gemini-api/docs/api-key.

7. Optional: enable Ollama chat

Ollama runs an AI model on the local computer. It is independent of Gemini and the Python packages. It is optional, and the AI Code Generator's project-generation flow does not require it.

Install Ollama

Download the installer for the operating system from ollama.com/download. Follow Ollama's current installation instructions for your system.

Download the model this project expects

The current Ollama chatbot code uses the model name phi and connects to Ollama at localhost:11434. In a new terminal, run:

```
ollama pull phi
ollama list
```

The model download can take time and use substantial disk space. Local model speed depends on the computer. If the pull command reports that the model name is not available, check the current Ollama model library and the model name configured by the project.

Start Ollama and use chat

Ollama Desktop may start its local service automatically. If needed, start the Ollama service in a separate terminal and leave it running:

```
ollama serve
```

Keep the AI Code Generator running in its own terminal as well. In the app, choose the Ollama chatbot option. The chatbot should connect to http://localhost:11434.

> **Gemini vs Ollama**
> Gemini sends chat requests to Google's service and requires a key and internet. Ollama runs a model locally, requires the separate Ollama installation and model download, and can be slower or unavailable on lower-memory machines. You can skip both to generate projects.

Ollama connection check

If the app says Ollama is not reachable, confirm Ollama is installed, the model pull completed, and the Ollama service is running. Try ollama list in another terminal. Do not change the Python app's server port to fix an Ollama connection error; they are separate services.

8. Daily use and project files

Start the app on a later day

Open a terminal, move into the cloned project, activate the environment, then start Flask from backend:

```
cd $HOME\Documents\ai-code-generator
.\.venv\Scripts\Activate.ps1
cd backend
python app.py
```

For macOS/Linux, use cd ~/Documents/ai-code-generator, then source .venv/bin/activate, then cd backend and python app.py. If using Gemini, set GEMINI_API_KEY in the same terminal before python app.py.

Where things are saved

Path	Contents
generated_projects/	Generated app folders and ZIP files. These files are local and can be deleted if no longer needed.
data/generation_history.json	A small local record of generated project metadata and prompts.
data/app.log	Local application log used to help diagnose errors.
data/snippet_metadata.json	The source catalog that connects reusable snippets to their code files. Keep this file with the project.
.venv/	Installed Python packages for this clone. Recreate it by following section 4 if deleted.

Change the port if 5000 is already in use

The server uses port 5000 by default. In PowerShell, set a different port before starting it:

```
$env:FLASK_RUN_PORT = "5001"
cd backend
python app.py
```

Then open http://127.0.0.1:5001. In Command Prompt, use set FLASK_RUN_PORT=5001. In macOS/Linux, use export FLASK_RUN_PORT=5001.

Get a later update from GitHub

Stop the app, return to the repository folder (not backend), and run git pull. Then activate .venv and restart the app. If you edited files yourself, save a backup first; updates can conflict with local edits.

```
cd $HOME\Documents\ai-code-generator
git pull
```

9. Troubleshooting

Problem	Try this
git is not recognized	Install Git, close and reopen the terminal, then run git --version.
py or python is not recognized	Install Python from python.org, close and reopen the terminal, then run py --version on Windows or python3 --version on macOS/Linux.
No module named flask (or another module)	Activate .venv, return to the repository root, and run pip install -r requirements.txt. Confirm the terminal prompt includes (.venv).
Cannot open the page / connection refused	Confirm python app.py is still running in the terminal. Use the exact address printed there, normally http://127.0.0.1:5000.
Template or module not found	Stop the server. From the repository root, activate .venv, enter backend with cd backend, and start python app.py there.
Port 5000 is already in use	Set FLASK_RUN_PORT to 5001 as shown in section 8, restart, and open port 5001 in your browser.
Gemini says a key is required	Set GEMINI_API_KEY in the same terminal session before starting python app.py. Check for missing quotes or extra spaces.
Gemini request fails despite a key	Check internet access, key validity, API access, account quota, and provider status. Do not paste the key into a support message.
Ollama is not reachable	Start Ollama, finish ollama pull phi, and verify the service is listening on localhost:11434.
pip install fails or stalls	Check internet access and free disk space. Upgrade pip with python -m pip install --upgrade pip, then retry. Keep the complete error output.

A clean reinstall of the Python environment

If the environment is damaged, stop the app and remove only the .venv folder inside the cloned project. Then recreate it using section 4. This does not remove generated_projects or other project files.

```
# From the repository root, Windows PowerShell
Remove-Item -Recurse -Force .venv
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If you need help, send the exact command you ran, your operating system, python --version, and the full error text. Never include an API key or password.

10. First-run checklist

- I installed Python and Git and confirmed both commands work in a new terminal.
- I cloned https://github.com/pranavparmaj/ai-code-generator and entered the ai-code-generator folder.
- I created .venv, activated it, and installed requirements.txt without errors.
- I started the app from the backend folder with python app.py.
- I opened http://127.0.0.1:5000 and generated a small sample project.
- I understand that the server terminal must stay open while I use the app.
- If I want Gemini chat, I set my own GEMINI_API_KEY only in my terminal environment.
- If I want Ollama chat, I installed Ollama, downloaded phi, and left its service running.

If every required step is complete

The project-generation workflow is running locally. Optional AI chat services can be configured later; they are not required for the main generation flow.

Project folder map

Folder / file	Purpose
backend/	Flask app and Python logic. Run app.py from this directory.
frontend/	Web interface templates, styles, and JavaScript.
templates/ and snippets/	Reusable generated-project templates and code fragments.
data/snippet_metadata.json	Lookup data for the source snippets.
utils/	ZIP export and diagram helper code.
requirements.txt	Python dependencies installed into .venv.
README.md	Short project setup reference.

Important usage boundary

This guide gets the app running for local use. The repository is not configured as a production service. Do not expose the built-in development server or live project preview to the public internet. Generated applications are starter code and should be reviewed and secured before any real use.

Official links and quick commands

These links are included so your friend can find trusted installation and configuration pages:

- Python downloads
- Git downloads
- GitHub Desktop
- Google AI Studio API keys
- Gemini API key safety guidance
- Ollama downloads
- AI Code Generator repository

Windows quick-start, after Python and Git are installed

```
cd $HOME\Documents
git clone https://github.com/pranavparmaj/ai-code-generator.git
cd ai-code-generator
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cd backend
python app.py
```

Open http://127.0.0.1:5000. Stop with Ctrl+C.

macOS/Linux quick-start, after Python and Git are installed

```
cd ~/Documents
git clone https://github.com/pranavparmaj/ai-code-generator.git
cd ai-code-generator
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cd backend
python app.py
```

This manual describes the repository state at the time it was prepared. Software installers, service requirements, and provider account policies can change; use the linked official pages for current details.